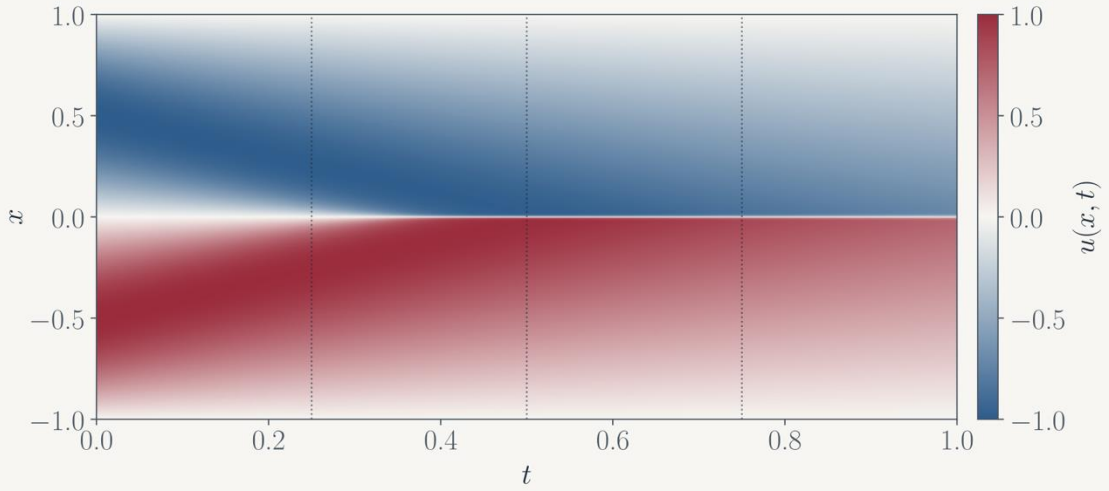
COURSE INTRODUCTION

Physics-Informed Neural Networks

Solving partial differential equations with deep learning

Nassim RAZAALY



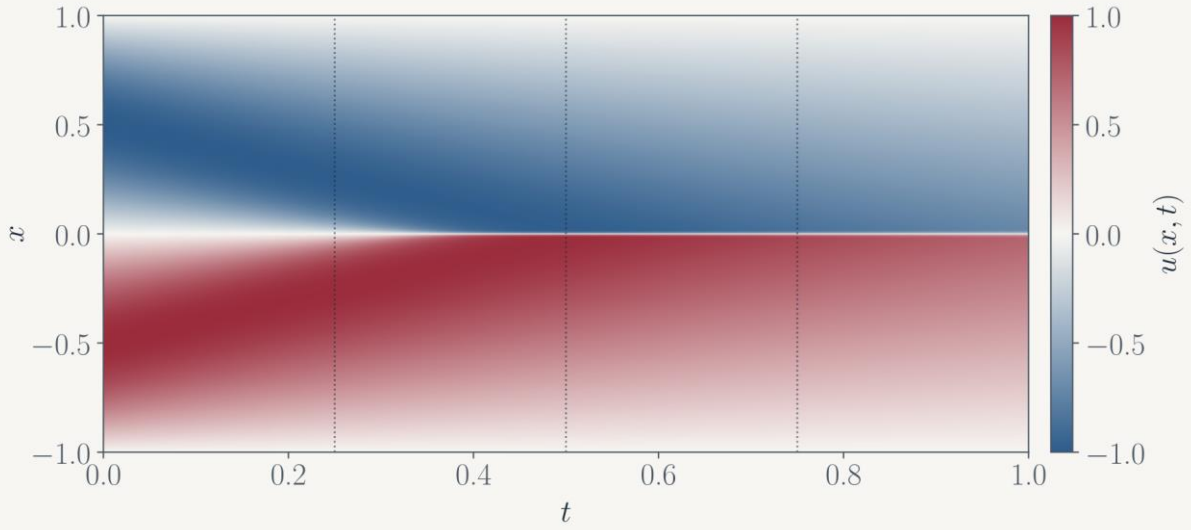
8 sessions of 1h30

First version: 14/10/26 · Last updated: 14/10/26

Goal: solve the Burgers equation with a neural network that knows the physics

$$\frac{\partial u}{\partial t} + u\,\frac{\partial u}{\partial x} = \nu\,\frac{\partial^2 u}{\partial x^2}$$

Ingredient	Value
Domain	$x \in [-1, 1], \quad t \in [0, 1]$
Initial condition	$u(x, 0) = -\sin(\pi x)$
Boundary conditions	$u(-1, t) = u(1, t) = 0$
Fixed parameter	$\nu = 0.01/\pi \approx 0.0032$



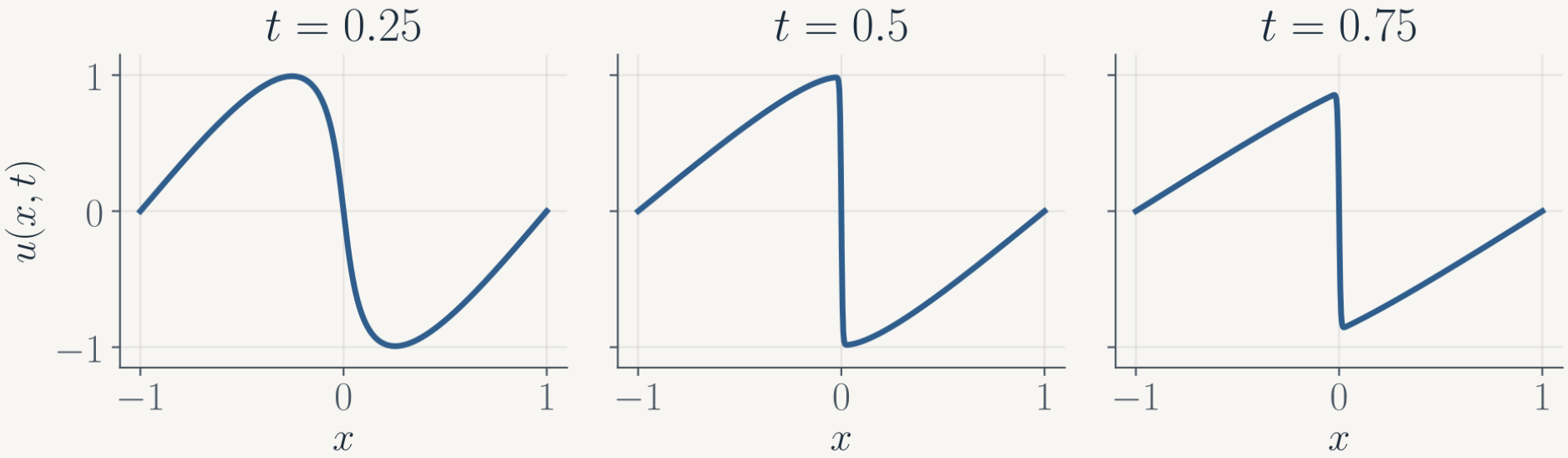
Exact solution u(x, t): a steep front forms at x = 0 around t ≈ 0.3

Forward problem ν known: find u(x, t)

Inverse problem find the PDE parameters from measurements of u

Benchmark of Raissi, Perdikaris & Karniadakis, J. Comput. Phys. 378 (2019) · solution by the Cole–Hopf transform

Why Burgers? A simple equation with a steep front



Nonlinear

The advection term steepens the profile

Multiscale

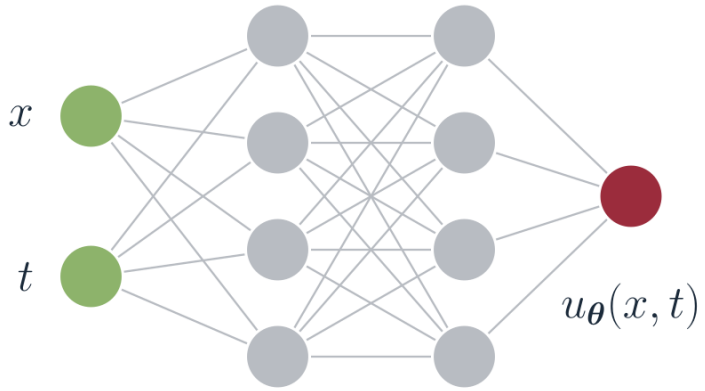
Small viscosity ν: a front of width ≈ ν

Known solution

Exact reference to measure the error

How a PINN works: a network trained to satisfy the PDE

Neural network



parameters $\boldsymbol{\theta}$, activation σ

Automatic differentiation

$u_{\boldsymbol{\theta}}$ $\quad \dfrac{\partial u_{\boldsymbol{\theta}}}{\partial t}$

$\dfrac{\partial u_{\boldsymbol{\theta}}}{\partial x}$ $\quad \dfrac{\partial^2 u_{\boldsymbol{\theta}}}{\partial x^2}$

exact derivatives w.r.t. inputs

Physics-informed loss

$$r_{\boldsymbol{\theta}} = \frac{\partial u_{\boldsymbol{\theta}}}{\partial t} + u_{\boldsymbol{\theta}}\frac{\partial u_{\boldsymbol{\theta}}}{\partial x} - \nu\frac{\partial^2 u_{\boldsymbol{\theta}}}{\partial x^2}$$

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{PDE}} + \mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{BC}}$$

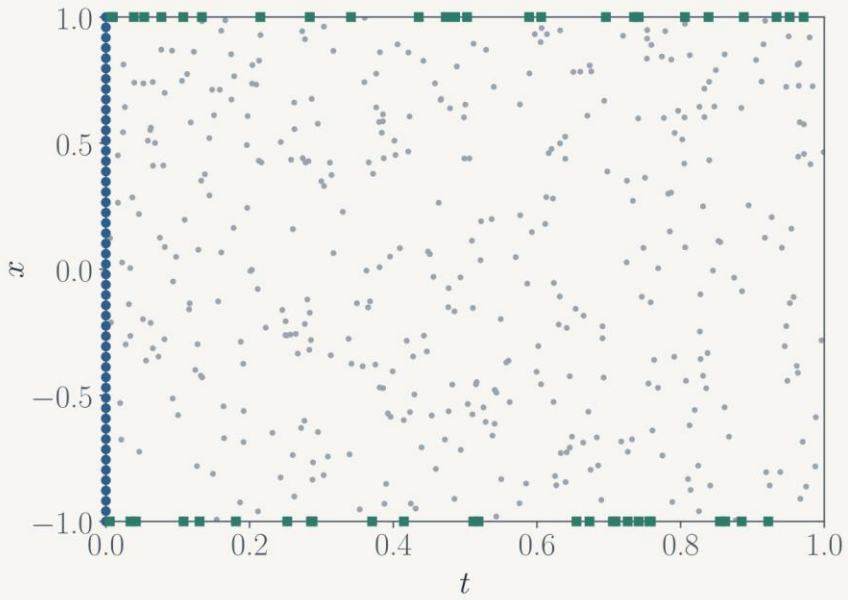
$\mathcal{L}_{\text{PDE}}$: residual $r_{\boldsymbol{\theta}} = 0$ inside

$\mathcal{L}_{\text{IC}}$: $u_{\boldsymbol{\theta}}(x,0) = -\sin(\pi x)$

$\mathcal{L}_{\text{BC}}$: $u_{\boldsymbol{\theta}}(\pm 1, t) = 0$

optimizer (Adam, L-BFGS): update $\boldsymbol{\theta}$ to minimize $\mathcal{L}(\boldsymbol{\theta})$

The loss: residual, initial and boundary conditions



$$\mathcal{L}(\boldsymbol{\theta}) =$$

PDE residual
at collocation points inside the domain

$$\frac{1}{N_r}\sum_{j=1}^{N_r}\left|r_{\boldsymbol{\theta}}(x_j,t_j)\right|^2$$

+ initial condition
at points on $t=0$

$$\frac{1}{N_0}\sum_{j=1}^{N_0}\left|u_{\boldsymbol{\theta}}(x_j,0)+\sin(\pi x_j)\right|^2$$

+ boundary conditions
at points on $x=\pm 1$

$$\frac{1}{N_b}\sum_{j=1}^{N_b}\left|u_{\boldsymbol{\theta}}(\pm 1,t_j)\right|^2$$

No solution data needed: only the equation, its initial condition and its boundary conditions.

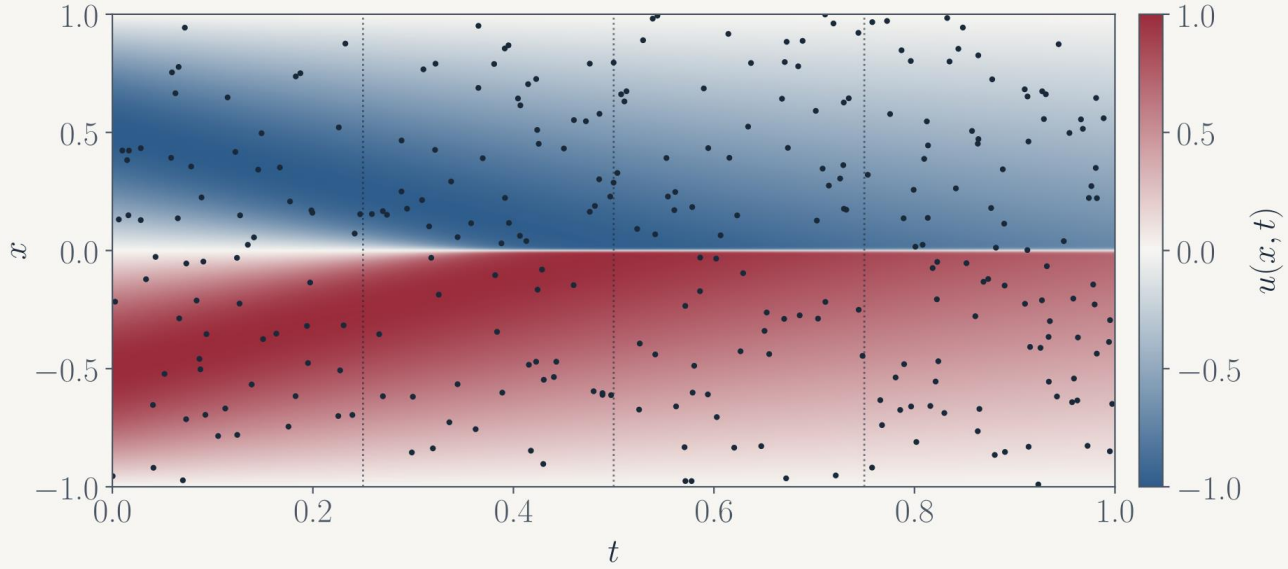
Identify the parameters of the PDE from data

$$\frac{\partial u}{\partial t} + \lambda_1 \, u \, \frac{\partial u}{\partial x} = \lambda_2 \, \frac{\partial^2 u}{\partial x^2}$$

Unknown: $\boldsymbol{\lambda} = (\lambda_1, \lambda_2)$

True values: $\lambda_1 = 1, \quad \lambda_2 = 0.01/\pi$

Data: scattered measurements of u (dots).



$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\lambda}) = \frac{1}{N_d} \sum_{j=1}^{N_d} \left| u_{\boldsymbol{\theta}}(x_j, t_j) - u_j^{\text{obs}} \right|^2 + \frac{1}{N_r} \sum_{j=1}^{N_r} \left| r_{\boldsymbol{\theta}, \boldsymbol{\lambda}}(x_j, t_j) \right|^2$$

θ and **λ** are learned together, by the same optimizer.

8 sessions of 1h30, alternating course and hands-on

#	Type	Content
1	Course	Course introduction · Introduction to ML: supervised learning, the multilayer perceptron
2	Hands-on	Introduction to PyTorch: tensors, autograd, a first MLP
3	Course	Training a network: loss, optimizers, mini-batches, hyperparameters
4	Hands-on	Learning a function f(x, y) applied to an image · Fourier features
5	Course	Introduction to PINNs: the forward Burgers problem
6	Hands-on	PINN for the forward Burgers problem
7	Course	Inverse problems with PINNs · better optimizers (L-BFGS and recent methods)
8	Hands-on	Inverse Burgers problem · open exercises: other PDEs, better optimizers

Course materials: what to use, and when

Slides — for the course

Course introduction
Session 1

Introduction to ML
Sessions 1 and 3

Introduction to PINNs
Sessions 5 and 7

Notebooks — for hands-on

Introduction to PyTorch
Session 1

Learning f(x, y): image, Fourier features
Session 4

PINN: forward Burgers
Session 6

Inverse problem, open exercises
Session 8

Lecture notes — to read after

Introduction to ML: supervised learning
Sessions 1 to 4 · full derivations, worked examples

Introduction to PINNs
Sessions 5 to 8

Prerequisites

Mathematics

- Linear algebra: vectors, matrices, norms
- Multivariable calculus: gradient, chain rule
- PDEs: initial and boundary conditions

Programming

- Python and NumPy
- Jupyter notebooks
- No prior PyTorch or machine-learning experience needed

Setup

- Notebooks on **github.com/nrazaaly/coursePINNs**
- Local: Python 3, PyTorch, NumPy, Matplotlib
- Or Google Colab: click the **Open in Colab** badge, nothing to install

Before session 2: open the GitHub page and check that a notebook runs, locally or on Colab. The first hands-on session starts directly with PyTorch.

The full course is online

github.com/nrazaaly/coursePINNs

Slides · lecture notes · notebooks

Public link

Material is added as the course goes. Everything is online on the last day.

Removed the day after.

No account needed

PyTorch notebooks: run them locally, or on Colab with one click.

Open in Colab